Ders 24 Çalışma Özeti

Ders 24 Çalışma Özeti

Genel Konular

- Metin arama (string matching / pattern matching) problemi ve kullanım alanları.
- Kaba kuvvet (brute force) yöntemi ve en kötü durum karmaşıklığı.
- Boyer-Moore-Horspool algoritması: kaydırma tablosu (shift table) ve eşleştirme sözde kodu.
- Boyer-Moore algoritması: kötü karakter kaydırması (bad symbol shift) ve iyi sonek kaydırması (good suffix shift).
- BAOBAB örneğiyle Boyer-Moore'un adım adım çözümü.
- Rabin-Karp algoritması: modüler aritmetik ve kayan özet (rolling hash).
- Algoritmaların karmaşıklık karşılaştırması.

Hocanın Özellikle Vurguladığı Kısımlar

- Kaba kuvvet yöntemi, eşleşme ihtimali olmayan konumları da tek tek dener. Bu "körebe gibi" bir yaklaşımdır.
- Daha iyi yöntemlerin fikri: uyuşmazlık olduğunda örüntüyü bir değil, mümkün olduğunca fazla kaydırmak.
- Boyer-Moore ön işlem ister. Ön işlem metin üzerinde değil, örüntü üzerinde yapılır. Örüntü (m) metinden (n) çok kısa olduğu için maliyet düşüktür.
- Boyer-Moore karşılaştırmayı sağdan sola, Knuth-Morris-Pratt soldan sağa yapar.
- Kaydırma tablosunu doldururken örüntünün son karakteri hesaba katılmaz.
- Kötü karakter, metindeki uyuşmayan karakterdir; örüntüdeki karakter değildir. Örnekte boşluk karakterinin unutulmaması gerekir.
- İki kaydırma birlikte kullanılırsa daha büyük olan seçilir. Hiç eşleşme yoksa ($k = 0$) yalnız kötü karakter kaydırması kullanılır.
- Horspool'un gerçeklemesi kolaydır. Performansı Boyer-Moore'a yakındır. Bu yüzden tercih edilebilir.

Kısa Tekrar Notları

- Gösterim: metin T (uzunluk n), örüntü P (uzunluk m), alfabe Σ.
- Kaba kuvvet: en kötü durumda $O(nm)$. Tekrarlı metin ve örüntülerde (ör. `AAAAB` içinde `AAAAA...`) bu durum oluşur.
- Horspool kaydırma tablosu:
  - Örüntüde olmayan karakter için kaydırma m.
  - Örüntünün ilk $m-1$ karakterinden biri için kaydırma: o karakterin son geçtiği yerden örüntü sonuna olan uzaklık.
- Boyer-Moore kötü karakter kaydırması: $d_1 = \max(t_1(c) - k, 1)$. c uyuşmayan metin karakteri, k eşleşen karakter sayısıdır.
- Boyer-Moore iyi sonek kaydırması d_2: eşleşen sonekin örüntüde bir önceki geçişine göre hesaplanır.
- Toplam kaydırma: $k = 0$ ise $d = d_1$; $k > 0$ ise $d = \max(d_1, d_2)$.
- Rabin-Karp: örüntünün özeti (hash) ile metindeki m uzunluklu pencerelerin özetleri karşılaştırılır. Özet eşitse karakterler tek tek kontrol edilir.

Detaylı Açıklamalar

Metin Aramanın Kullanım Alanları

- Kelime işlemcilerde bul/değiştir.
- Belge ve kod benzerliği, tezlerin özgünlük kontrolü.
- Biyoenformatikte DNA dizilimi araması.
- Veri madenciliği, metin sınıflandırma, konuşma tanıma.
- Metin büyüdükçe (milyonlarca belge) zaman karmaşıklığı kritik hâle gelir.

Kaba Kuvvet (Brute Force) Yöntemi

- Örüntü metnin başına hizalanır. Karakterler soldan sağa karşılaştırılır.
- Uyuşmazlık olursa örüntü bir sağa kaydırılır.
- Dış döngü $n - m$ konumuna kadar gider; sonrasında örüntü sığmaz.
- İç döngü örüntünün boyu kadar döner.

```
int search(char *pat, char *txt) {
  int M = strlen(pat);
  int N = strlen(txt);
  int i, j;
  for (i = 0; i <= N - M; i++) {
    for (j = 0; j < M; j++)
      if (txt[i + j] != pat[j])
        break;
    if (j == M)
      return i;       /* eşleşme bulundu */
  }
  return N;           /* bulunamadı */
}
```

- En kötü durum $O(nm)$: tekrarlı örüntüde son karakter hep uyuşmaz. Örnek: `AAAAAAAAAB` metninde `AAAAB` aramak. Her konumda 4 karakter eşleşir, 5. uyuşmaz.

Daha İyi Kaydırma Fikri

- `ABRA` aranırken ilk karakterler eşleşip sonra uyuşmazlık çıkarsa örüntüyü bir sonraki `A`'nın altına kaydırmak yeterlidir.
- `ABCD` içinde `XYZ` aranırken `X` metinde hiç yoksa örüntü doğrudan metnin sonuna kaydırılabilir.
- Bunun için hangi karakterin örüntüde nerede olduğunu önceden bilmek gerekir. Bu bilgi ön işlemle elde edilir.

Boyer-Moore-Horspool Algoritması

Karşılaştırma örüntünün sonundan başlar. Uyuşmazlık olunca kaydırma miktarı, örüntünün o anki son karakterinin hizasındaki metin karakterine (c) göre belirlenir.

Dört durum (`BARBER` örüntüsü, $m = 6$):

Durum	Açıklama	Kaydırma
1	c örüntüde yok (ör. `S`)	Örüntü boyu kadar: 6
2	c örüntüde var (ör. `B`), son karakter değil	c'nin örüntüdeki en sağdaki geçişi hizalanır
3	c yalnız örüntünün son karakteri olarak var (`LEADER` içindeki `R`)	Örüntü boyu kadar
4	c son karakterle eşleşti ve örüntüde başka yerde de var	O başka geçiş hizalanır

`BARBER` için kaydırma tablosu:

Karakter	A	B	E	R	Diğerleri
Kaydırma	4	2	1	3	6

Tablonun doldurulması: - Önce tüm karakterlere $m = 6$ yazılır. - Örüntü soldan sağa, son karakter hariç gezilir. Her karaktere $m - 1 - j$ yazılır. - `B` ($j = 0$): 5 - `A` ($j = 1$): 4 - `R` ($j = 2$): 3 - `B` ($j = 3$): 2. Önceki 5'in üzerine yazılır. - `E` ($j = 4$): 1 - Son karakter `R` ($j = 5$) işlenmez. Bu yüzden `R`'nin değeri 3 kalır. - Soldan sağa gezilir ki aynı karakterin en sağdaki geçişi önceki değeri ezsin.

```
ALGORITHM ShiftTable(P[0..m-1])
  for i <- 0 to size-1 do Table[i] <- m
  for j <- 0 to m-2 do Table[P[j]] <- m - 1 - j
  return Table
```

```
ALGORITHM HorspoolMatching(P[0..m-1], T[0..n-1])
  ShiftTable(P[0..m-1])
  i <- m - 1                  // örüntünün sağ ucunun konumu
  while i <= n - 1 do
    k <- 0                    // eşleşen karakter sayısı
    while k <= m - 1 and P[m-1-k] = T[i-k] do
      k <- k + 1
    if k = m
      return i - m + 1        // eşleşme
    else i <- i + Table[T[i]]
  return -1
```

Boyer-Moore Algoritması

İki tablo kullanılır.

Kötü karakter kaydırması (bad symbol shift): - $d_1 = \max(t_1(c) - k, 1)$. - t_1 Horspool'daki tablodur. c metindeki uyuşmayan karakterdir. k uyuşmazlıktan önce eşleşen karakter sayısıdır. - Örnek: `BARBER`'ın `ER`'i eşleşti, `S` uyuşmadı. $d_1 = 6 - 2 = 4$. - Örnek: `ER` eşleşti, `A` uyuşmadı. $d_1 = 4 - 2 = 2$. - $\max(\ldots, 1)$ sebebi: $t_1(c) - k$ sıfır ya da negatif çıkabilir. Örüntüyü sola kaydırmak anlamsızdır. En az 1 kaydırılır.

İyi sonek kaydırması (good suffix shift): - Eşleşen sonekin (k karakter) örüntüde bir önceki geçişine göre kaydırma yapılır. - Sonekin başka geçişi yoksa, sonekin bir parçasının örüntünün önekiyle (prefix) eşleşmesine bakılır. - `ABCBAB` için:

k	Sonek	d_2
1	`B`	2
2	`AB`	4
3	`BAB`	4
4	`CBAB`	4
5	`BCBAB`	4

- Öğrenciye sorulan: `C` yerine `A` olsaydı ($ABABAB$) `AB`'nin bir önceki geçişi 2 geride olurdu, $d_2 = 2$.
- Önek ve sonek örnekleri (`ABCBAB`):
  - Önekler: `A`, `AB`, `ABC`, `ABCB`, `ABCBA`.
  - Sonekler: `B`, `AB`, `BAB`, `CBAB`, `BCBAB`.

Algoritmanın adımları: 1. Kötü karakter tablosunu oluştur. 2. İyi sonek tablosunu oluştur. 3. Örüntüyü metnin başına hizala. 4. Sağdan sola karşılaştır. Tüm karakterler eşleşirse dur. Uyuşmazlıkta: - $k = 0$ ise $d = d_1$. - $k > 0$ ise $d = \max(d_1, d_2)$. - Yalnız birini de gerçekleyebilirsiniz; hata değildir. İkisi birlikte daha hızlı kaydırır. - $k = 0$ iken iyi sonek kullanılmaz; eşleşen sonek olmadığı için yanıltıcı kaydırma üretir.

BAOBAB Örneği

Örüntü `BAOBAB` ($m = 6$).

Kötü karakter tablosu (t_1):

Karakter	A	B	O	Diğerleri (boşluk dahil)
t_1	1	2	3	6

- `A`'nın ilk değeri 4'tür, sonra ikinci `A` ile 1 olur.
- `B`'nin ilk değeri 5'tir, sonra 2 olur. Son `B` hesaba katılmaz.

İyi sonek tablosu (d_2):

k	Sonek	d_2
1	`B`	2
2	`AB`	5
3	`BAB`	5
4	`OBAB`	5
5	`AOBAB`	5

- `AB` örüntüde başka yerde geçmiyor. `B`'nin önünde `O` var. Yalnız baştaki `B`, sonekin son harfiyle hizalanabilir. Bu yüzden $d_2 = 5$.

Metin: `BESS_KNEW_ABOUT_BAOBABS`

Adım	Durum	d_1	d_2	Kaydırma
1	İlk karşılaştırmada uyuşmazlık (`K` ile `B`), $k = 0$	$t_1(K) - 0 = 6$	Kullanılmaz	6
2	`AB` eşleşti, boşlukta uyuşmazlık, $k = 2$	$6 - 2 = 4$	5	$\max(4, 5) = 5$
3	`B` eşleşti, boşlukta uyuşmazlık, $k = 1$	$6 - 1 = 5$	2	$\max(5, 2) = 5$
4	Tüm karakterler eşleşti			Bulundu

- Sınıfta yapılan hata: uyuşmayan karakter olarak örüntüdeki `B` alındı. Doğrusu metindeki boşluk karakteridir. Bir öğrenci uyardı.
- Tüm geçişler isteniyorsa algoritma eşleşmeden sonra devam edecek şekilde değiştirilmelidir.

Rabin-Karp Algoritması

- Karakter karşılaştırması yerine "parmak izi" (fingerprint) karşılaştırması yapar.
- Görüntü işlemedeki şablon eşleme (template matching) ile benzetme yapıldı: tüm pikselleri değil, ayırt edici özellikleri karşılaştırmak.
- Örnek (sayılarla): örüntü `26535`, mod 997.
  - $26535 \bmod 997 = 613$.
  - Metindeki her 5 haneli pencerenin mod 997 değeri hesaplanır. Örnek: $31415 \bmod 997 = 508$.
  - Değerler eşit değilse örüntü bir sağa kaydırılır.

  - Değerler eşitse karakterler tek tek kontrol edilir. Farklı sayılar aynı kalanı verebilir (çakışma).
- Kayan özet: yeni pencerenin değeri bir öncekinden hesaplanır.
  - $41592 \to 15926$: $41592 - 4 \cdot 10000 = 1592$, $1592 \cdot 10 + 6 = 15926$.
  - Her adımda baştan çarpma yapmak gerekmez.
- Karakterler ASCII değerlerine çevrilip aynı işlem uygulanır.
- Horner kuralı gibi her karakteri farklı bir kuvvetle (R^i) çarpmak, `AB` ve `BA` gibi dizilerin aynı özeti üretmesini önler.
- Özet (hash) fonksiyonları $O(1)$ arama ve kriptografide de kullanılır. Ayrıntısı Algoritma Analizi dersinde.

Karmaşıklık Karşılaştırması

N karakterli metinde M karakterli örüntü arama maliyeti:

Algoritma	Tipik	En kötü
Kaba kuvvet	$1.1N$ karakter karşılaştırması	MN karakter karşılaştırması
Karp-Rabin	$3N$ aritmetik işlem	$3N$ aritmetik işlem (rastgele)
KMP	$1.1N$ karakter karşılaştırması	$2N$ karakter karşılaştırması
Boyer-Moore	yaklaşık N/M karakter karşılaştırması	$3N$ karakter karşılaştırması

- Deneysel grafikler:
  - Örüntü uzadıkça kaba kuvvet ve KMP'nin performansı pek değişmiyor.
  - Boyer-Moore ise örüntü uzadıkça hızlanıyor.
- Boyer-Moore ile Horspool'un sonuçları birbirine yakın. Horspool'un gerçeklemesi kolay olduğu için tercih edilebilir.

Ders sonrası notlar

- Boyer-Moore'un en kötü durumda $3N$ karşılaştırması, örüntünün metinde geçmediği durum içindir. Tüm geçişler aranırken temel Boyer-Moore en kötü durumda $O(nm)$'ye çıkabilir. Bunu önlemek için Galil kuralı gibi iyileştirmeler vardır.
- Hoca KMP'nin en kötü durumunu Boyer-Moore'a "hemen hemen aynı" diye yorumladı. Tabloya göre KMP'nin en kötü durumu ($2N$) Boyer-Moore'unkinden ($3N$) daha iyidir. Boyer-Moore'un üstünlüğü tipik durumdadır (N/M).
- Hoca $\max(t_1(c) - k, 1)$ ifadesindeki 1'i açıklarken $t_1(c) = 3$, $k = 5$ örneğinde "3 - 5, 2 çıkacaktı" dedi. Doğrusu $3 - 5 = -2$'dir. Negatif kaydırma anlamsız olduğu için $d_1 = 1$ alınır.
- C dilinde ayrı bir `string` tipi yoktur. Metin, `'\0'` ile biten `char` dizisi olarak tutulur.
- Rabin-Karp'ta özet karşılaştırması bir konumu $O(1)$'de eler. Çakışma olduğunda karakter karşılaştırması gerektiği için en kötü durum yine $O(nm)$'dir. Mod için büyük bir asal seçmek çakışma olasılığını düşürür.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.